

REBOT B601-DM CAPSTONE · WEEK OF 14-20 SEP 2026

Finding the crack, then reaching it.

A zero-shot crack-segmentation front-end for robotic concrete inspection
— built, reviewed and proven end to end on the GPU this week.

rebot_crack_vision · OpenCrack nnU-Net v2 · RTX 3090

THE PROBLEM

A robot arm can only inspect a crack it can locate

The capstone goal is a reBot B601-DM arm that follows a crack across a concrete surface with a sensor on its tool — bridge and structure inspection without a human tracing the defect by hand. That needs perception first. Everything downstream — depth, path, trajectory — is meaningless if the pixels marked "crack" are wrong.

This week's question: can a publicly released crack model work *zero-shot* on our imagery, or does the capstone have to fund a labelling and training effort of its own?

THE CANDIDATE

Model	OpenCrack nnU-Net v2, 2D
Weights	268 MB · 92.5 M params · fold 0
Trained on	49,531 images
Reported	0.539 IoU · 0.641 cIoU
Licence	CC-BY-4.0
Our cost	zero training runs

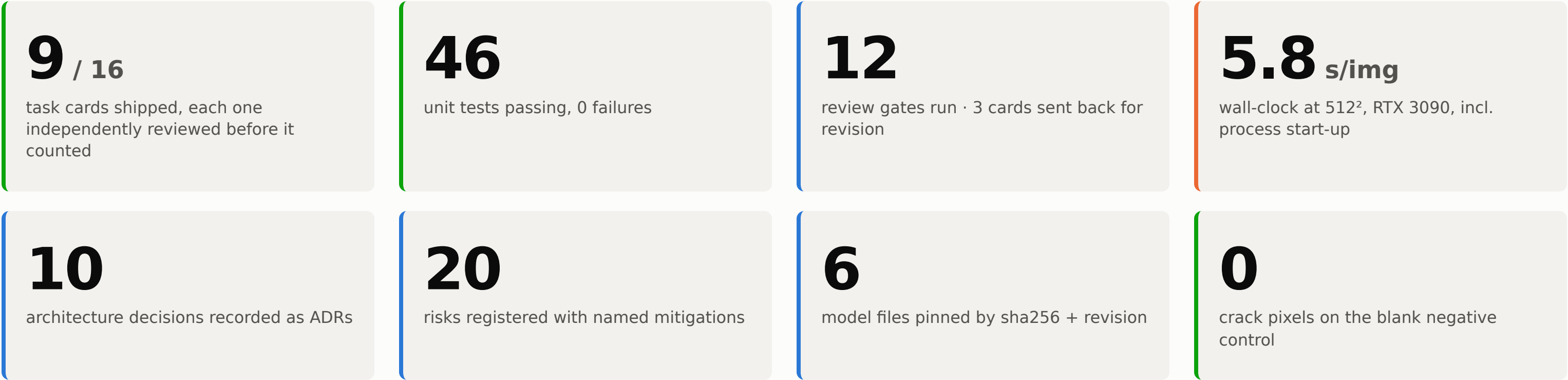
no fine-tuning

no labelling

no training GPU-hours

WHAT SHIPPED

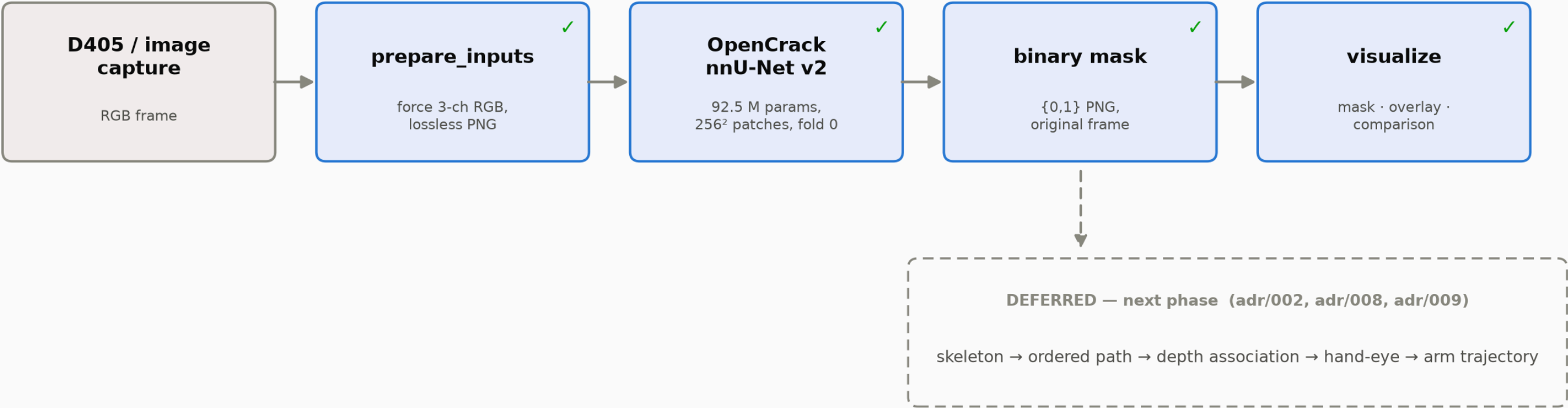
The week in numbers



The pipeline, and the contract underneath it

One frozen contract holds the whole chain together

Nothing resizes, crops or pads. A (row, col) in the mask is the same (row, col) in the original frame — and later, in the aligned depth frame.



✓ = shipped and independently reviewed this week

The frame-invariant rule is the load-bearing decision: break it once and a mis-registered trajectory shows up months later, far from the cause.

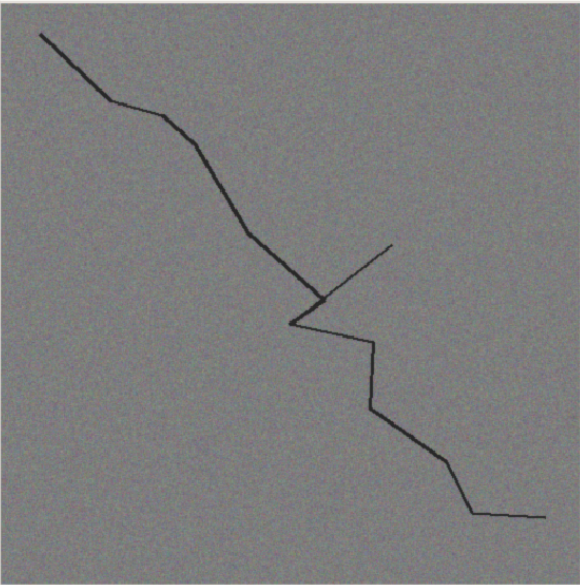
RESULT

Image in, centerline out — on real GPU inference

OpenCrack nnU-Net v2 · zero-shot, end-to-end on an RTX 3090

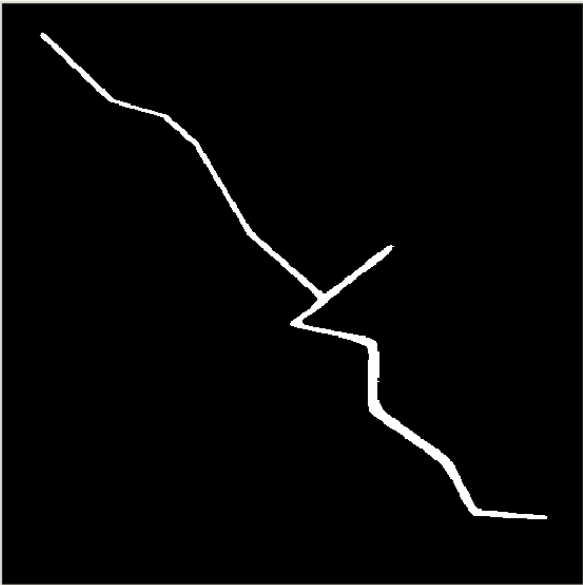
Every stage stays in the original image frame — no resize, crop or pad — so a (row, col) here indexes the aligned depth frame later.

1 · INPUT



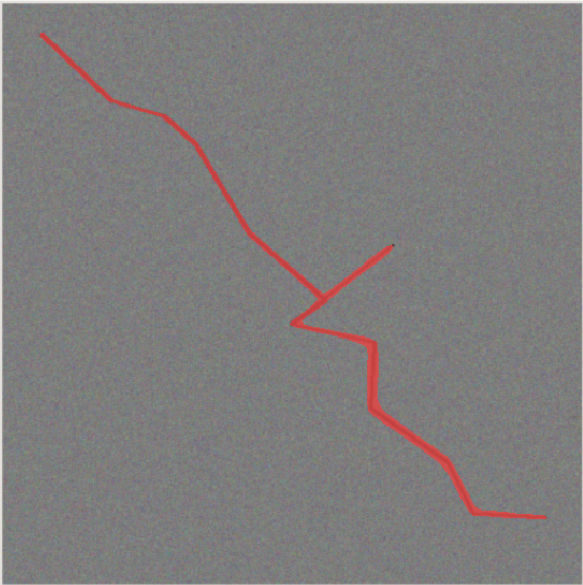
512x512 RGB, unmodified frame

2 · MASK



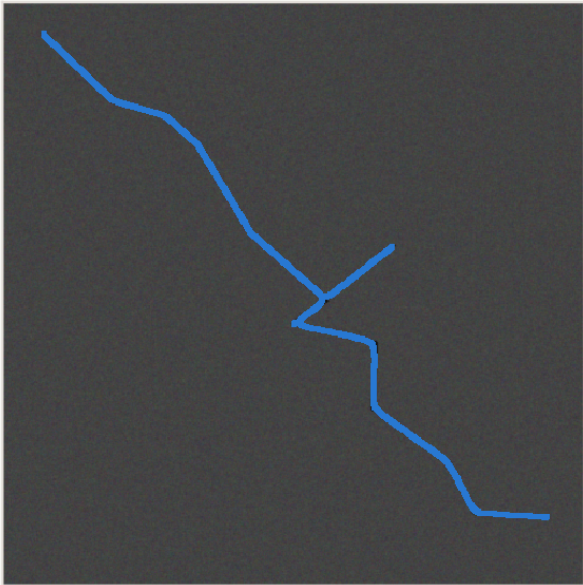
binary {0,1} PNG · 4,577 crack px (1.75 %)

3 · OVERLAY



α-blended review render

4 · CENTERLINE



1-px skeleton, 657 px (thickened for print)

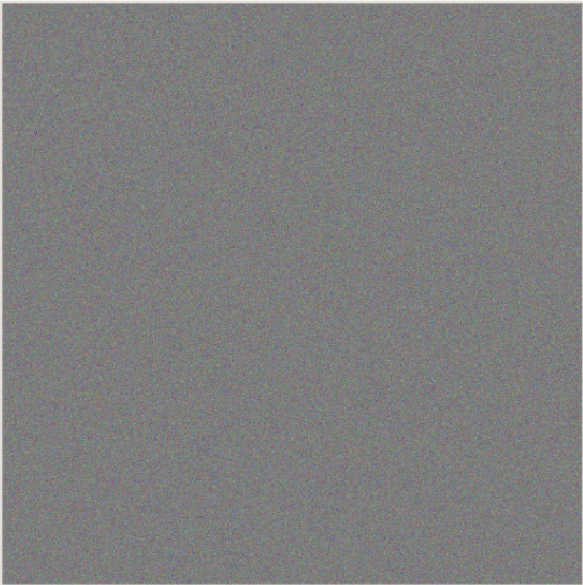
Level-2 smoke test, run on demand: 4,577 crack pixels (1.75 % of frame), reduced to a 657-pixel one-pixel-wide centerline.

Every run has to fail the blank frame too

The smoke test asserts both directions, every run

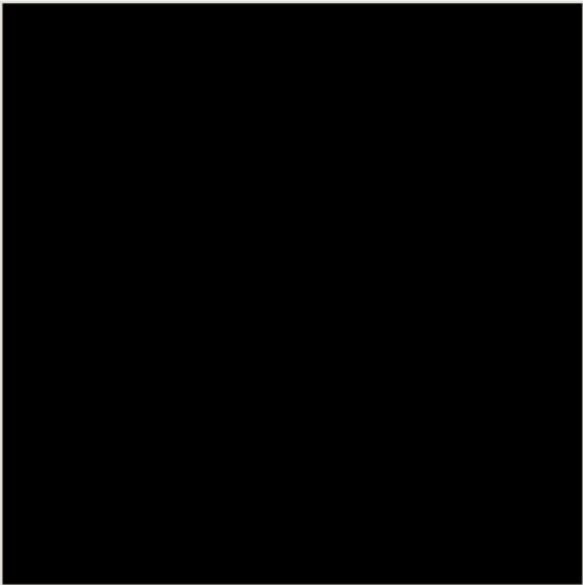
A pipeline that always fires looks identical to a working one on crack imagery alone. The blank frame is what makes the positive case mean something.

NEGATIVE CONTROL · input



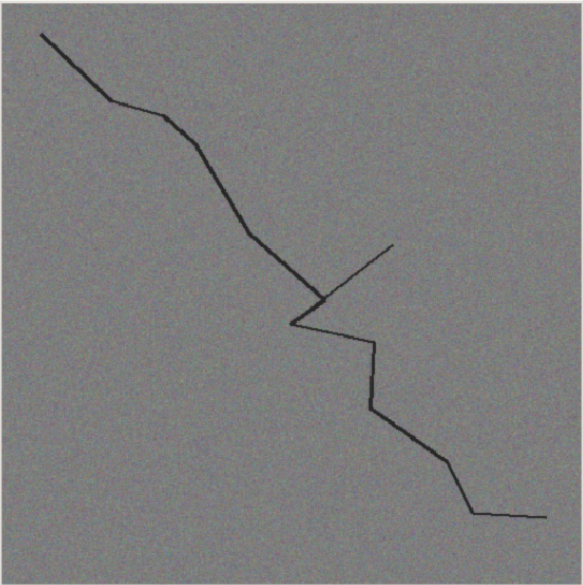
textured concrete, no crack present

NEGATIVE CONTROL · output



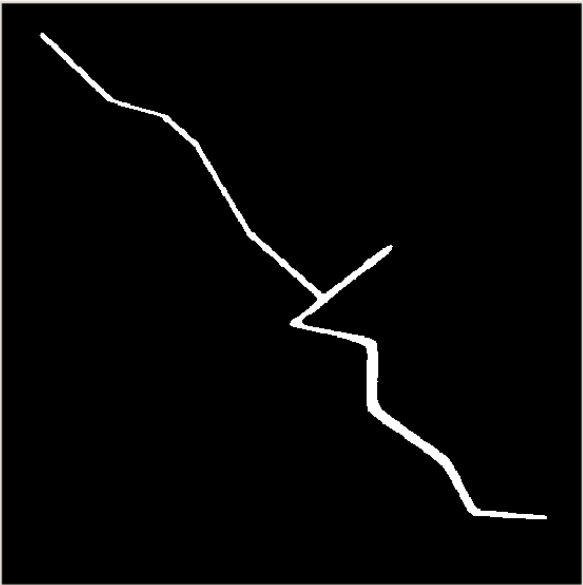
0 crack px — zero false positives

POSITIVE CASE · input



same texture, one synthetic crack

POSITIVE CASE · output



4,577 crack px — crack recovered

A pipeline stuck in the "on" position looks identical to a working one when you only ever show it cracks.

Three failures worth designing against

R-01 · ENVIRONMENT

ROS poisons the interpreter. The shell profile puts ROS Humble's Python 3.10 packages on the path of a 3.11 process — pure-Python packages shadow real ones, compiled ones throw opaque ABI errors.

`env.sh` scrubs `PYTHONPATH`, filters `/opt/ros` out of the loader path, and the verifier *fails the build* if a ROS entry ever returns.

R-02 · SHARED GPU

The card is not ours alone. Other projects on this workstation park multi-GB servers on GPU 0 without warning.

An OOM prints a five-step degradation ladder as literal next commands instead of a traceback — `--not_on_device`, fewer workers, no TTA, downscale, CPU. No task may ever kill another project's GPU process.

R-03 · SILENT MISREAD

Predictions are {0,1}, not {0,255}. The mask PNG looks solid black in any viewer. The natural reaction — "it detected nothing, let me rescale it" — corrupts the data.

Documented, logged once per run, binarised as `> 0` everywhere, and pinned by a test so it can never drift.

PROCESS

Nothing counted as done until someone else checked it

9 of 16 task cards shipped and independently approved this week

Each card: written spec → implementation agent → independent adversarial review → approve or send back. Three cards came back for revision before they were approved.

■	TC-001	Scaffolding & git hygiene	APPROVED
■	TC-002	Python 3.11 env + env.sh	APPROVED
■	TC-003	Environment verification	APPROVED
■	TC-004	OpenCrack model acquisition	APPROVED
■	TC-005	nnU-Net wiring + validation	APPROVED
■	TC-006	Synthetic smoke test (L2)	APPROVED
■	TC-007	Input prep + naming contract	APPROVED
■	TC-008	Batch inference runner	APPROVED
■	TC-009	Visualization pipeline	APPROVED
■	TC-010	Skeletonization utility	READY
■	TC-012	RealSense software validation	READY
■	TC-011	One-command run_test.sh	QUEUED
■	TC-013	D405 capture utility	QUEUED
■	TC-014	Integration + hygiene tests	QUEUED
■	TC-015	Evaluation-manifest tooling	QUEUED
■	TC-016	Docs & reproducibility	QUEUED

Spec → implementation → adversarial review that re-runs every acceptance command → approve or send back. Three cards were sent back.

WHAT REVIEW CAUGHT

The bugs that would have surfaced much later

A corrupt model file, downloaded clean. `plans.json` had a duplicated JSON document appended to it — every parse threw `Extra data: line 218`. Caught by semantic validation of the model tree, not by a crash three cards later.

A card marked complete that wasn't. TC-005 scored 7/10 at review and was rejected. Approving it would have flipped downstream work to "ready" on a model tree that could not be loaded.

Two upstream traps, found in planning. The model card documents a filename that does not exist in the repository, and nnU-Net defaults to five folds and a checkpoint this model does not ship. Both are now pinned in config and asserted by tests.

Every approval re-derived from the filesystem. Reviews re-ran the acceptance commands rather than reading the implementer's report — including a full real-model GPU run, with GPU memory checked back to its idle baseline afterwards.

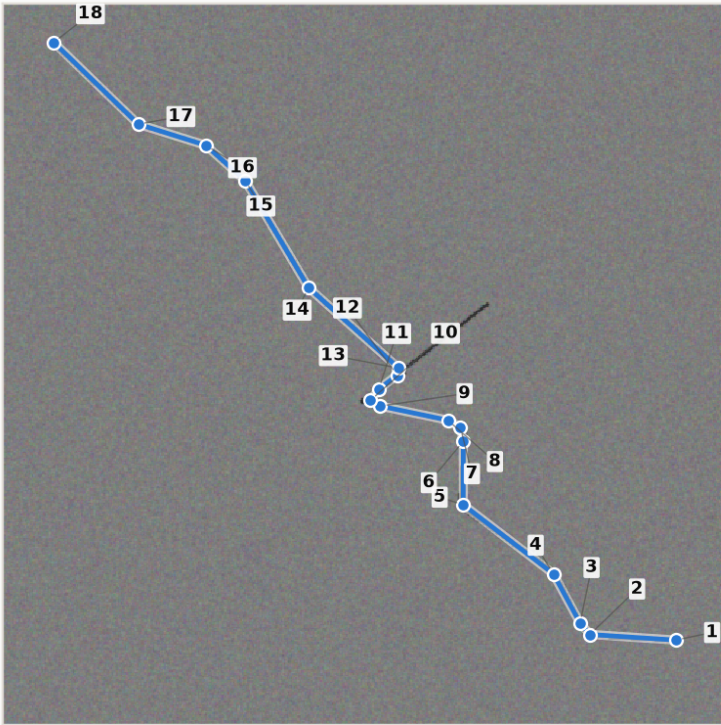
DEMO · NOT YET A VALIDATED CAPABILITY

Following the vision output into the arm

From a crack mask to a joint-space motion

18 vision waypoints → 20-pose IK solve · 24.3 s at a constant 2 cm/s tool speed · max tracking error 19.367 mm · every joint stays inside its limit.

1 · VISION PATH



18 ordered centerline waypoints on the source frame

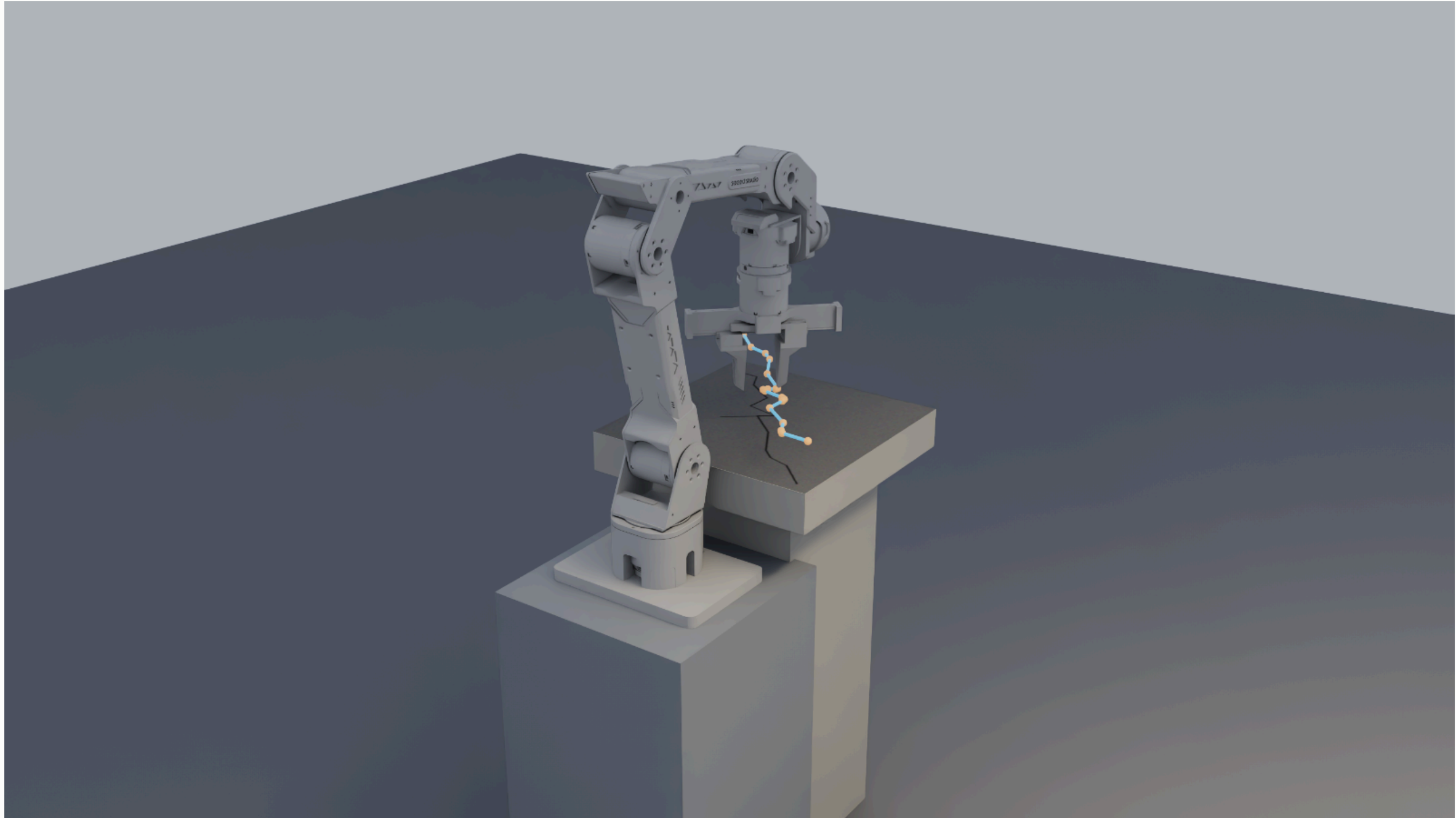
2 · SOLVED JOINT TRAJECTORY



18 ordered centerline waypoints → a 20-pose IK solve for the B601-DM: the 18 real inspection points converge to **0.026 mm mean error**, every joint inside its limit, 24.3 s at a constant 2 cm/s. The two hover transitions (before/after the pass) land 5.7–19.4 mm short even after a full recovery search — reported, not hidden; see docs/TECHNICAL_APPROACH.md. The surface pose and the camera→robot transform are *declared*, not calibrated — but the coupon's exact placement was chosen from a measured IK-reachability sweep, not guessed.

DEMO · NOT YET A VALIDATED CAPABILITY

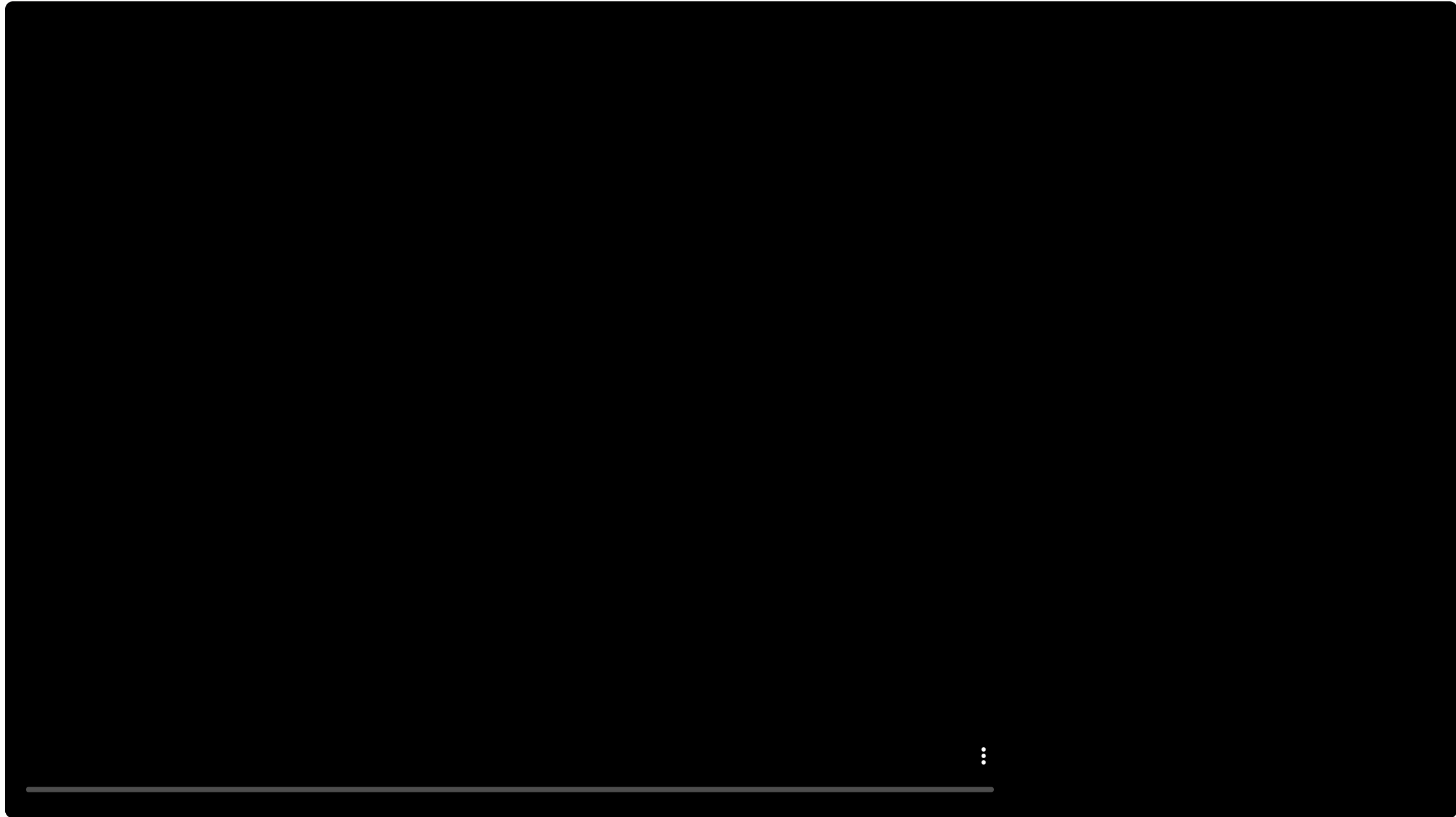
The same path, in Isaac Sim



The real B601-DM URDF, a concrete coupon carrying the actual image the model ran on, and the vision-derived path in the scene — rendered headless on the same RTX 3090 that ran the segmentation.

DEMO · NOT YET A VALIDATED CAPABILITY

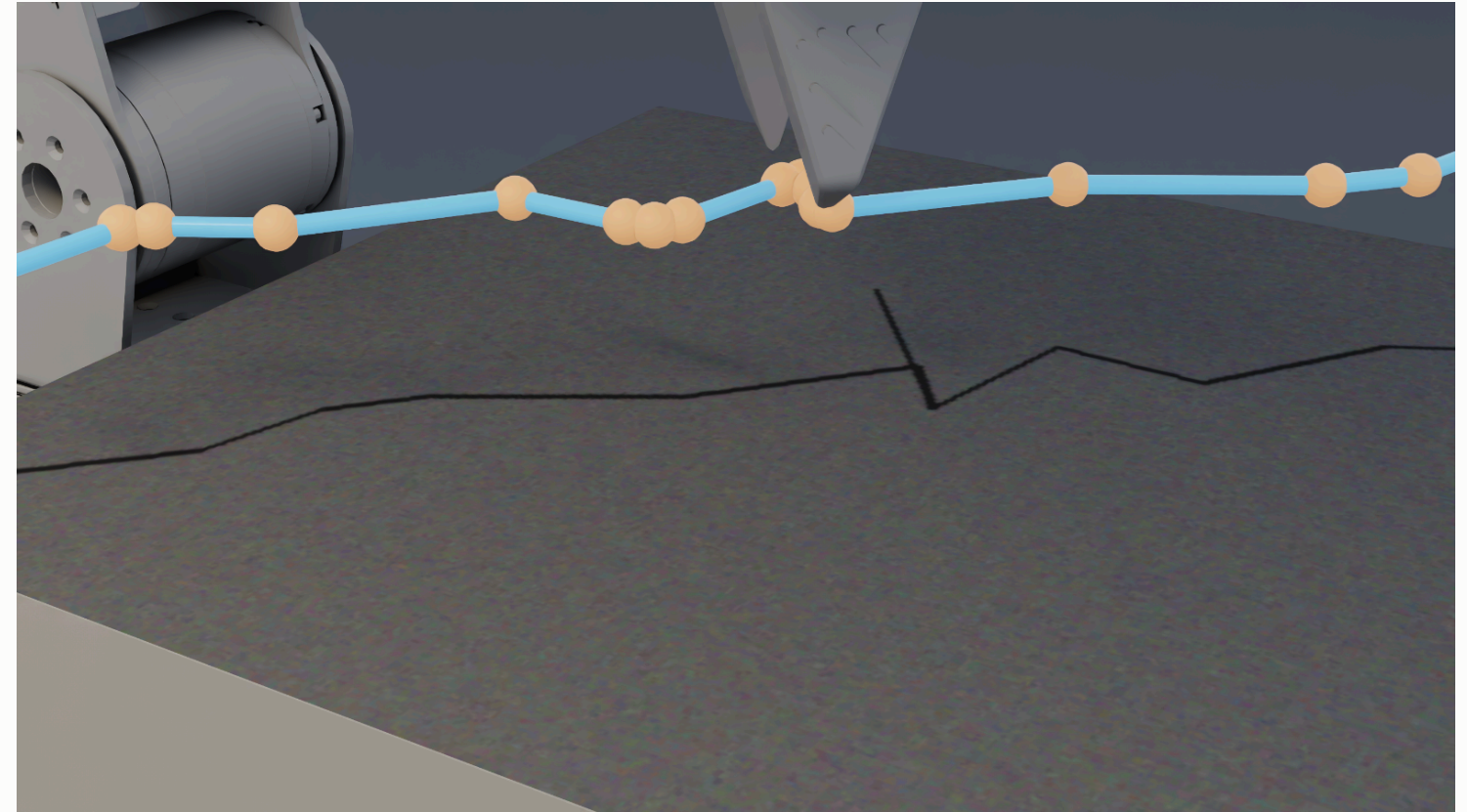
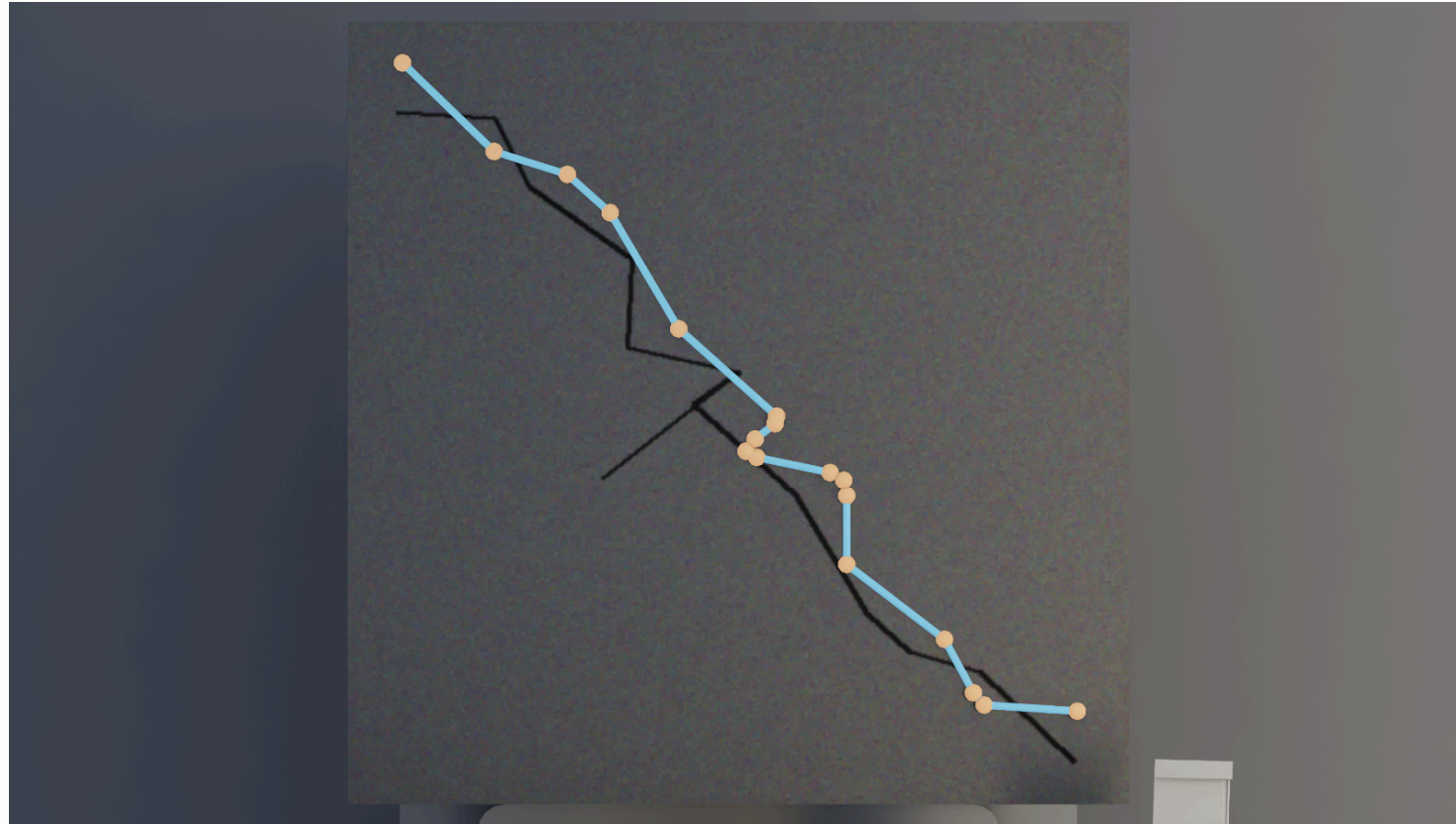
Watching it move



Every joint angle here is the actual solved IK, keyframe to keyframe — only the motion *between* keyframes is interpolated, and playback runs at 2× real time (both stated on-screen). Watch the wrist: it stays genuinely vertical across the whole 18-point inspection pass, and visibly tips at the very end — the one waypoint that didn't fully converge, shown, not cut.

DEMO · NOT YET A VALIDATED CAPABILITY

Does the path actually land on the crack?



Orthographic, camera dead-centre above the coupon, path flattened to zero standoff so there is no parallax to hide behind — this is the actual proof shot. Four colour-coded fiducials (not shown here) confirm the texture's corners land exactly on the coupon's geometric corners. A few millimetres of visible fuzz remain — sphere/tube size and ordinary texture-filtering blur, not a coordinate bug — documented in [presentation/sim/README.md](#).

What is measured, and what is still declared

MEASURED

- Real nnU-Net v2 inference on an RTX 3090, timed per image
- Mask values, frame shape and channel handling, asserted by tests
- Zero false positives on the blank control frame
- Model tree integrity by sha256 against a pinned revision
- 46 unit tests across naming, input preparation and visualization

DECLARED, NOT YET MEASURED

- **Zero-shot quality on real D405 imagery** — the headline question; no camera is attached yet, so nothing is claimed
- Depth at the crack — a crack is often narrower than the stereo window
- Camera→robot extrinsics — the demo transform is chosen, not hand-eye calibrated
- IoU/Dice against labels — Level-3 evaluation, deliberately out of this phase

The demo that follows the vision into the arm is scaffolding for this talk, not a validated capability.

NEXT

What the remaining seven cards unlock

CARD	UNLOCKS
TC-010	skeletonization as a first-class stage
TC-011	the whole chain behind one command
TC-012/13	D405 RGB + aligned depth capture
TC-014	integration and contract tests
TC-015/16	evaluation manifests, reproducibility

Then the real experiment. A D405 test plan is already written: a distance sweep, and negative controls chosen to be hard — construction seams, stains, aggregate, scratches, marker lines. The point is to characterise the failure surface deliberately rather than discover it by accident on the robot.

Only after that does depth projection, hand-eye calibration and a real arm trajectory become honest work rather than a demo.

TAKEAWAY

A perception front-end you can trust the rest of the robot to

Works

Public crack weights run zero-shot, end to end, on our hardware — no training, no labelling, no licence problem

Pinned

Environment, model bytes, mask convention and image frame are all asserted by something that fails loudly

Honest

The headline question — real D405 imagery — is scheduled and instrumented, not quietly assumed

Everything here is reproducible from the repository: pinned requirements, a launcher that scrubs the environment, a hash-verified model tree, and a smoke test that proves the GPU path in about fourteen seconds.